Python Cheat Sheet

Für alle die tiefer einsteigen wollen 🚀

① List Comprehensions

Kompakte Kurzschreibweise um Listen zu erstellen. Ersetzt for-Schleifen mit append() in einer einzigen Zeile. Optional mit Filterbedingung am Ende.

```
# Klassisch – umständlich
ergebnis = []
for i in range(5):
    ergebnis.append(i * 2)

# Kompakt – eine Zeile
ergebnis = [i * 2 for i in range(5)]
# [0, 2, 4, 6, 8]

# Mit Filterbedingung
gerade = [i for i in range(10) if i % 2 == 0]
# [0, 2, 4, 6, 8]
```

💡 Lesbarkeit hat Vorrang. Verschachtelte Comprehensions besser als for-Schleife schreiben.

② Lambda, map & filter

Lambda erzeugt einzeilige anonyme Funktionen ohne def. map() wendet eine Funktion auf jedes Element einer Liste an, filter() behält nur bestimmte Elemente.

```
# Lambda: Mini-Funktion ohne def
quadrat = lambda x: x ** 2
quadrat(5)   # 25

addiere = lambda a, b: a + b
addiere(3, 4)  # 7

# map: Funktion auf alle anwenden
doppelt = list(map(lambda x: x*2, [1, 2, 3]))
# [2, 4, 6]

# filter: nur bestimmte behalten
```

💡 List Comprehensions sind oft lesbarer als map() und filter().

③ *args & **kwargs

*args fängt beliebig viele positionelle Argumente als Tuple auf. **kwargs fängt benannte Argumente als Dictionary. Beide zusammen ermöglichen maximale Flexibilität.

```
# *args: beliebig viele Werte
def summe(*args):
    return sum(args)

summe(1, 2, 3, 4)  # 10

# **kwargs: beliebig viele Keys
def info(**kwargs):
    for k, v in kwargs.items():
        print(f"{k}: {v}")

info(name="Anna", alter=25)
```

💡 * entpackt Listen, ** entpackt Dicts, auch beim Funktionsaufruf nutzbar.

④ Klassen & OOP

Klassen bündeln Daten (Attribute) und Verhalten (Methoden) in einem Objekt. self verweist auf die aktuelle Instanz, __init__ ist der Konstruktor der beim Erstellen aufgerufen wird.

```
class Tier:
    # Klassenvariable (geteilt)
    anzahl = 0

    def __init__(self, name, alter):
        self.name = name      # Attribut
        self.alter = alter
        Tier.anzahl += 1

    def vorstellen(self):   # Methode
        return f"Ich bin {self.name}"
```

💡 __str__ für print(), __repr__ für die offizielle Darstellung, __eq__ für Vergleiche.

Python Cheat Sheet

Für alle die tiefer einsteigen wollen 🚀

⑤ Vererbung

Eine Kindklasse erbt alle Methoden und Attribute der Elternklasse und kann sie überschreiben. super() ruft die Elternmethode auf ohne den Namen hart zu kodieren.

```
class Tier:
    def __init__(self, name):
        self.name = name

    def sprechen(self):
        return "..."

class Hund(Tier):    # erbt von Tier
    def __init__(self, name, rasse):
        super().__init__(name)
        self.rasse = rasse
```

💡 isinstance(obj, Klasse) prüft auch Elternklassen. issubclass() prüft Klassenhierarchie.

⑥ Decorators

Verpacken eine Funktion in eine andere ohne ihren Code zu ändern. Mit @-Syntax angewendet. Ideal für Logging, Timing, Zugriffsschutz oder Caching.

```
def logger(func):
    def wrapper(*args, **kwargs):
        print(f"Aufruf: {func.__name__}")
        ergebnis = func(*args, **kwargs)
        print("Fertig.")
        return ergebnis
    return wrapper

@logger
def addieren(a, b):
    return a + b
```

💡 from functools import wraps verwenden um Metadaten der Originalfunktion zu erhalten.

⑦ Generators

Generators erzeugen Werte erst bei Bedarf statt alles auf einmal in den Speicher zu laden. yield statt return pausiert die Funktion und merkt sich den Zustand.

```
# Generator-Funktion mit yield
def quadrate(n):
    for i in range(n):
        yield i ** 2   # pausiert hier

for q in quadrate(5):
    print(q)   # 0, 1, 4, 9, 16

# next() manuell aufrufen
gen = quadrate(3)
next(gen)   # 0
next(gen)   # 1
```

💡 Ideal für große Datensätze, CSV-Zeilen oder Streams die nicht komplett ins RAM passen.

⑧ File I/O & Pathlib

with open() schließt die Datei automatisch, auch bei Fehlern. Pathlib macht Pfade plattformunabhängig und lesbar, ohne Strings mit / oder \ zu verketten.

```
# Lesen
with open("datei.txt", encoding="utf-8") as f:
    inhalt = f.read()       # gesamter Text
    zeilen = f.readlines()  # Liste der Zeilen

# Schreiben / Anhängen
with open("out.txt", "w") as f:
    f.write("Hallo!\n")          # überschreibt

with open("out.txt", "a") as f:
    f.write("Neue Zeile\n")      # anhängen
```

💡 Modi: "r" lesen, "w" schreiben (überschreibt), "a" anhängen, "rb"/"wb" für Binärdaten.

Python Cheat Sheet

Für alle die tiefer einsteigen wollen 🚀

⑨ Dataclasses

@dataclass generiert __init__, __repr__ und __eq__ automatisch. Spart viel Boilerplate bei einfachen Datenklassen. field() ermöglicht komplexere Defaults.

```
from dataclasses import dataclass, field

@dataclass
class Spieler:
    name: str
    punkte: int = 0
    items: list = field(default_factory=list)

s = Spieler("Anna")
print(s)
# Spieler(name='Anna', punkte=0, items=[])
```

💡 *Niemals list oder dict als Default direkt angeben, immer field(default_factory=list) nutzen.*

⑩ Dict & Set Tricks

Dictionaries und Sets haben viele nützliche Methoden die oft übersehen werden. Besonders .get(), .setdefault() und Mengenoperationen auf Sets sparen viel Code.

```
# Sicherer Zugriff mit Fallback
d = {"a": 1, "b": 2}
d.get("c", 0)          # 0 statt KeyError
d.setdefault("c", [])  # nur setzen wenn fehlt

# Dicts mergen (Python 3.9+)
merged = d1 | d2
d1 |= d2               # in-place

# Über Keys und Values
for k, v in d.items():
    print(k, v)
```

💡 *Counter aus collections zählt Häufigkeiten: Counter("aabbc") → {'a':2, 'b':2, 'c':1}.*

⑪ f-String Formatting

f-Strings können weit mehr als nur Variablen einsetzen. Sie unterstützen Ausdrücke, Zahlenformatierung, Ausrichtung und seit Python 3.8 auch Debug-Ausgaben.

```
name = "Anna"
preis = 9.5
menge = 3

# Ausdruck direkt einsetzen
print(f"Gesamt: {preis * menge:.2f} €")
# Gesamt: 28.50 €

# Debug-Modus (Python 3.8+)
x = 42
print(f"{x=}")          # x=42
```

💡 *f-Strings sind schneller als .format() und lesbarer als %-Formatierung.*

⑫ Context Manager

with-Blöcke garantieren Aufräumcode auch bei Fehlern. Eingebaut für Dateien, Locks und Verbindungen. Mit contextlib eigene Context Manager einfach erstellen.

```
# Klassisch: with open()
with open("datei.txt") as f:
    inhalt = f.read()
# Datei automatisch geschlossen

# Mehrere auf einmal
with open("a.txt") as a, open("b.txt") as b:
    daten = a.read() + b.read()

# Eigener Context Manager
from contextlib import contextmanager
```

💡 *__enter__ und __exit__ manuell implementieren für vollständige Klassen-basierte Context Manager.*